Latest updates: https://dl.acm.org/doi/10.1145/3649476.3660369

RESEARCH-ARTICLE

Hardware Accelerated Quantized Hand Written Digit Recognition via High Level Synthesis

PAWAN ORAON, National Institute of Technology Rourkela, Rourkela, OR, India

SOUMYASHREE MANGARAJ, National Institute of Technology Rourkela, Rourkela, OR, India

AYAS KANTA SWAIN, National Institute of Technology Rourkela, Rourkela, OR, India

KAMALAKANTA MAHAPATRA, National Institute of Technology Rourkela, Rourkela, OR, India

Open Access Support provided by:

National Institute of Technology Rourkela

PDF Download
3649476.3660369.pdf
26 February 2026
Total Citations: 1
Total Downloads: 837

Published: 12 June 2024

Citation in BibTeX format

GLSVLSI '24: Great Lakes Symposium on VLSI 2024
June 12 - 14, 2024
FL, Clearwater, USA

Conference Sponsors:
SIGDA

GLSVLSI '24: Proceedings of the Great Lakes Symposium on VLSI 2024 (June 2024)
https://doi.org/10.1145/3649476.3660369
ISBN: 9798400706059

Hardware Accelerated Quantized Hand Written Digit Recognition via High Level Synthesis

Pawan Oraon
pawanoraon0608@gmail.com
National Institute of Technology Rourkela
Rourkela, Odisha, India

Soumyashree Mangaraj
National Institute of Technology Rourkela
Rourkela, India

Ayas Kanta Swain
National Institute of Technology Rourkela
Odisha, India

Kamalakanta Mahapatra
National Institute of Technology Rourkela
Rourkela, Odisha, India

ABSTRACT

Convolutional Neural Networks (CNNs) have demonstrated remarkable success in image recognition tasks, but their deployment on resource-constrained devices remains challenging due to their computational complexity and memory requirements. This abstract presents an overview of hardware accelerator implementations for CNN-based image recognition, focusing on techniques to optimize performance, energy efficiency, and resource utilization. Hardware accelerators such as Field-Programmable Gate Arrays (FPGAs) offer parallel processing capabilities that can exploit the inherent parallelism in CNN computations. Design considerations include optimizing memory access patterns, and minimizing communication overhead between processing elements. Techniques such as pipelining, unrolling, quantization, and network compression are employed to reduce the computational and memory footprint of CNN models without significantly compromising accuracy. Hardware-software co-design methodologies enable seamless integration of CNN inference engines with host systems, facilitating real-time image recognition applications. The computational time of the proposed CNN is lower compared to that of the recent research works. Additionally, the proposed hardware design exhibits reduced memory, power consumption, and resource utilization as compared with recent literarture for MNIST digit recognition with 98.9% accuracy.

CCS CONCEPTS

• **Hardware** → **Emerging technologies**; • **Emerging architectures**;

KEYWORDS

FPGA, Convolutional Neural Network, Hardware Accelerator, Python, PYNQ-Z2, High Level Synthesis, Pipelining, Unrolling

ACM Reference Format:
Pawan Oraon, Soumyashree Mangaraj, Ayas Kanta Swain, and Kamalakanta Mahapatra. 2024. Hardware Accelerated Quantized Hand Written Digit Recognition via High Level Synthesis. In *Great Lakes Symposium on VLSI 2024 (GLSVLSI '24), June 12–14, 2024, Clearwater, FL, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3649476.3660369

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only. Request permissions from owner/author(s).
GLSVLSI '24, June 12–14, 2024, Clearwater, FL, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0605-9/24/06
https://doi.org/10.1145/3649476.3660369

1 INTRODUCTION

Handwritten digit recognition, is a critical component of modern computer vision and machine learning systems, it has witnessed a great impact across various industries. This technology enables the automated interpretation and conversion of handwritten numerical data into digital format, its efficiency and accuracy in tasks ranging from mail sorting to educational assessment. Applications of handwritten digit recognition in banking and finance, where it do faster and more efficient check processing, to healthcare, where it facilitates the digitization of medical records. In this paper, the principles, methodologies, and optimization of handwritten digit recognition has been analysed from a hard-ware acceleration viewpoint.

Now a days FPGAs[11] are increasingly being utilized in Convolutional Neural Network (CNN) [16] implementations for image recognition due to their flexibility, parallel processing capabilities, and low power consumption. FPGAs are used in CNN for image recognition because it can exploit parallelism inherent in CNN operations such as convolutions, pooling, and fully connected layers. They can perform these operations concurrently across multiple processing elements, leading to significant speedups compared to traditional CPUs. FPGAs consume less power compared to GPUs and CPUs [9] while delivering comparable or even superior performance in certain applications. FPGAs can efficiently utilize on-chip memory resources to reduce the need for external memory access, thereby minimizing memory bandwidth requirements and improving overall performance.

Computation complexity along with high training speed of deep-CNN (DCNN) architectures demands for the specialized hardware accelerators such as FPGAs[9]. The deployment of DCNN models on resource constraint FPGA is possible by quantizing [6] the model weights and activation to lower precision formats (e.g., INT8 or INT4) targeting to reduce memory usage and computational requirements while maintaining performance. Optimization in algorithms tailored for specific hardware platforms to improve computational efficiency is another perspective of model implementation on FPGA. In this work we have tried to expedite computation and inference accuracy both in hardware and algorithmic viewpoint.

The contributions of this paper are listed below.

- A simplified architecture is proposed with reduced number of parameters, layers, and filters ensuing an optimized algorithm for the convolutional kernel for PYNQ-Z2 FPGA acceleration [5].
- An optimized algorithm for the proposed architecture with optimization techniques such as memory reuse, data batching, and memory compression are adapted to minimize memory footprint during training and inference.
- The efficiency, speed, power, and scalability of the proposed implemented CNN model is compared with recent research works, and manifested it as a more suitable model for deployment in various applications and environments with hardware accelerators such as FPGAs.

The structure of this paper is as follows: Related work is explained in Section 2. In Section 3, the proposed CNN IP architecture is designed in Python and Vitis HLS [4], with HLS optimization techniques such as pipelining, unrolling, array partitioning, and quantization being detailed. Section 4 presents experimental results, including a confusion matrix, results of resource utilization, and on-chip power consumption, along with the block design of the quantized CNN model. Section 5 encompasses the discussion and a comparison table of resource utilization. Finally, Section 6 describes the conclusion and future scope of the proposed work.

2 RELATED WORK

The paper[15] has hardware accelerated design for CNN on low memory 32-bit edge devices uses an increased number of filters, leading to a higher quantity of weights that must be stored in BRAM. Consequently, this results in greater memory consumption, increased power usage, and greater resource utilization. Additionally, the utilization of 32-bit floating point throughout the operations leads to increased memory space required for storing variables.

3 METHODOLOGY

In this work, we propose a new and simplified CNN-based architecture using a total of 8 filters across subsequent layers. We employ a (3,3) size weight matrix for convolution with a stride of 1, and utilize a (2,2) matrix with a stride of 2 for maxpooling. The architecture is evaluated on the MNIST [8] dataset, consisting of 10,000 grayscale handwritten digits (7000 for training and 3000 for testing). The CNN model was trained on Google Colab over 100 epochs, achieving an accuracy of 98.9%.

3.1 Architecture

The proposed CNN architecture for classifying MNIST [8] handwritten digits is illustrated in Fig 1. The architecture consists of two convolutional layers followed by subsampling layers for dimensional reduction of feature-map. This process iterates once more before the resulting features are flattened by a flattening layer and passed into the fully connected layer of the CNN model [10].

The proposed CNN architecture layer-wise summary is presented in Table 1. The total 3122 trainable parameters of the weighted layers: convolution and dense are pre-computed during training, and used in the process of inference.

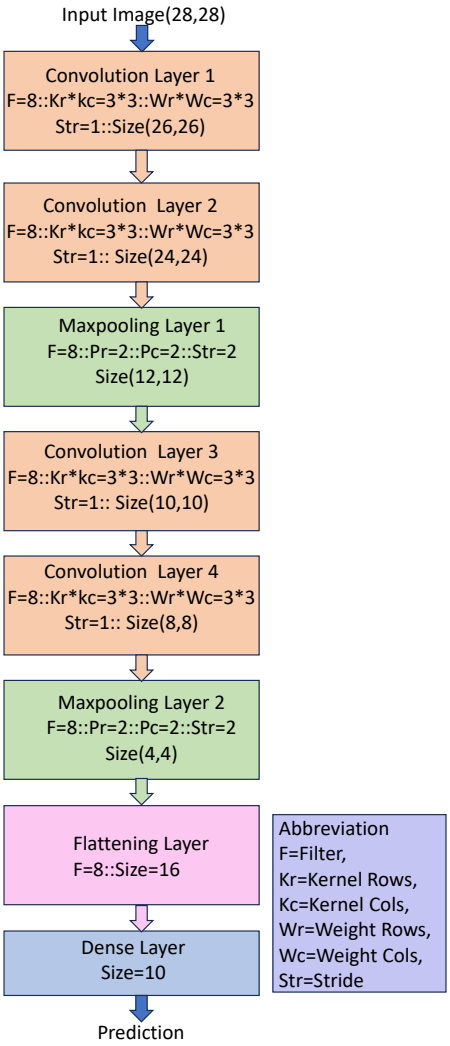

Figure 1: Architecture of Proposed CNN

Table 1: CNN Model Summary

Layer(Type)	Shape	Parameter
Convolutional Layer1	(26,26,8)	80
Convolutional Layer2	(24,24,8)	584
Maxpooling Layer1	(12,12,8)	0
Convolutional Layer3	(10,10,8)	584
Convolutional Layer4	(8,8,8)	584
Maxpooling Layer2	(4,4,8)	0
Flatten Layer	(128,8)	0
Dense Layer	(10,8)	1290

3.2 Workflow

The process begins with the development of a Convolutional Neural Network (CNN) architecture in Google Colab environment. Once the architecture is designed and validated, the trained weights are

extracted for subsequent use. After obtaining the fine tuned model along with its parameters, the next step is to proceed with the hardware implementation. The implementation on FPGA requires the bitstream file, and the generation of a bitstream necessitates the utilization of both source code and design constraint files, acquired through block design. This block design relies on dedicated IP obtained from VITIS HLS via High-Level Synthesis [12].

In VITIS HLS, the proposed CNN architecture is implemented in C++ [1], incorporating the trained weights as constraints. Subsequent synthesis and simulation of the code within VITIS HLS are followed by meticulous code optimization. The resultant RTL representation is exported as Intellectual Property (IP). This IP is then integrated into a block design, whereupon the design undergoes unwrapping.

Synthesis, implementation, and bitstream generation are performed on the unwrapped design. Subsequently, the generated bitstream and hardware file are uploaded onto the PYNQ Z2 board through Jupyter PYNQ framework. Finally, the overlay file derived from the hardware implementation is utilized for inference tasks, whereby input digits are fed to the system, and predictions are obtained. The abstract view of the workflow is presented in Fig 2.

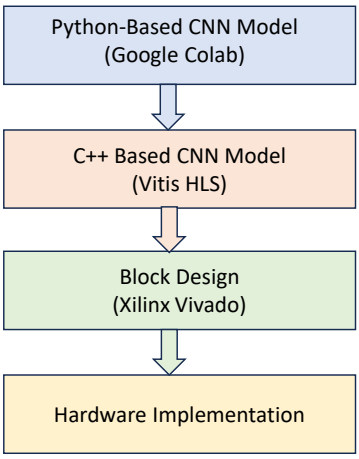

Figure 2: Software- Hardware combined workflow of proposed CNN

3.3 HLS Optimization

The CNN model is meticulously optimized using the VITIS HLS tool. Through HLS optimization techniques [2], our goal is to improve various performance metrics including throughput, resource utilization, and power consumption. The optimization in the design we proposed has ensured the accuracy of the synthesized hardware design with improvement in performance metrics. In the coding structure of HLS, a range of optimization strategies are employed in the program, including loop optimization, dataflow optimization, resource sharing, pipeline optimization, memory optimization, and directive optimization [2].

3.3.1 Pipeline Optimization. Here in this research, pipeline optimization technique has been used to improve the CNN model performance in the hardware. Pipeline in the inner loop of C++ code of Convolutional layer has been inserted which helps to decrease the overall latency of our CNN model.

3.3.2 Memory Optimization. The parameters and variables used in C++ code are stored in BRAM which are 32 bit floating point number, and so to decrease the memory usage Quantization [6] technique is used. ARM CORTEX A9 processor works on 32-bit, so, input and output are taken as 32 bits, and but the in between process data are not constrained to 32 bits. So, the trained weights are quantized from 32-bit floating point to 8-bit signed integer, and stored ".h" file. Quantization is applied in between parameters aiming to achieve less memory.

$Q(x) = \text{round}\left(\frac{x}{\text{scale_factor}}\right)$ where, Q(x) is the quantized value of x. Round denotes rounding the result to the nearest integer. Scale_factor is the scaling factor calculated based on the range of values in the dataset and the desired bit depth of the quantized representation.

$scale_factor = \left(\frac{\text{range}}{2^{n-1}-1}\right)$

where,
"n" is the number of bits (in this case, 8).
"range" is the range of values in my dataset.

3.3.3 Directive Optimization. Directive optimization in HLS refers to the process of fine-tuning and optimizing the directives and pragmas provided by HLS tools to guide the synthesis process effectively. These directives allow designers to specify constraints and optimizations to achieve the desired hardware implementation quality in terms of performance, resource utilization, and other metrics. There are various types of Directive Pragmas such as loop unrolling, pipeline iteration interval, and inlining.

3.3.4 Optimization in Convolution Algorithm. The convolution algorithm is presented in Listing 1, and followed with discussion.

```
#pragma HLS ARRAY_PARTITION
dim=1 type=complete variable=img_inn
int w_sum = 0;
conv_for_filters:
for(int f=0; f<FILTERS; f++){
conv_for_rows:
for(int r = 0; r < IMG_ROWS; r += 1) {
conv_for_cols:
    for(int c = 0; c < IMG_COLS; c += 1) {
    w_sum = 0.0;
krn_for_rows:
for(int kr = 0; kr < KRN_ROWS; ++kr) {
krn_for_cols:
for(int kc = 0; kc < KRN_COLS; ++kc) {
    w = conv_weights[f][kr][kc];
    pixel = img_in[r+kr][c+kc];
    w_sum += w * pixel;
        }
    }
conv_img[f][r][c]=relu(w_sum+conv_biases[f]);
        }
    }
}
```

Listing 1: C++ Code with HLS Directives

The outer loop('conv for filter') iterates over each filter. The middle loops ('conv for rows' and 'conv for cols') iterate over the rows

and columns of the input image, with a stride defined by 'POOL ROWS' and 'POOL COLS'. The innermost loop, there are two more nested loops ('krn for rows' and 'krn for cols') iterating over the rows and columns of the filter kernel. Inside the innermost loop, it performs the convolution operation by multiplying the corresponding elements of the 'filter weights' and the 'input image pixels' and accumulating the result in 'w sum'. This operation is repeated for each filter and each position in the image. Finally, it applies an activation function (reluu) to the accumulated sum and writes the result into 'conv img'. The code is using fixed-point arithmetic (ap_int<16>) for computations, and it seems to be optimized for FPGA implementation using pragmas (#pragma). The #pragma HLS PIPELINE II1 appears to indicate that the loop can be fully pipelined with an initiation interval of 1 cycle.

3.4 Dataflow

The dataflow presented in Fig. 3 between various blocks are controlled by a central control unit, which provides instructions on when and where to transmit data. The process begins with input data or images stored in external memory being fed into the input data buffer. From there, the data flows to the Convolution block via FIFO stream, while dedicated weights are simultaneously fetched from BRAM to the convolution block [11].

The convolution block performs the convolution operation, after which the output is summed with biases stored in BRAM.For second convolution, after convoluion of first filter the result stored in a buffer which then later add with the remaining filter results then after the final summation the total sum add with the biases and passed through the Relu function to retrive the necessary elements. Following the second convolution, the output proceeds to the max pooling block.

From here, the output of the max pooling block can either return to the input data buffer or be directed to the flatten block through FIFO stream, depending on the requirements. The flatten block then processes the output of the Max Pooling block and forwards it to the Dense Block through FIFO stream, ultimately yielding the final prediction.

4 EXPERIMENTAL RESULTS

In this section, we've explored several key metrics, including resource utilization, implemented design utilization, total on-chip power consumption, PS-PL interfacing, and implemented design and routing.

Table 2 describes the current resource utilization status in VITIS HLS. It provides guidelines on resource usage thresholds and compares them to our actual utilization. Based on the table, all resource utilization statuses are marked as 'OK', indicating that our usage remains within the recommended thresholds.

Fig. 4 displays the confusion matrix generated by our CNN Model. To construct this matrix, we utilized 7000 images for training and 3000 images for validation, resulting in a total of 10,000 images utilized for training and validation collectively. These images were processed over 100 epochs. Table 3 provides an overview of the total resources utilized by our model. The extensive use of variables in our C++ code contributes to the occurrence of BRAM utilization.

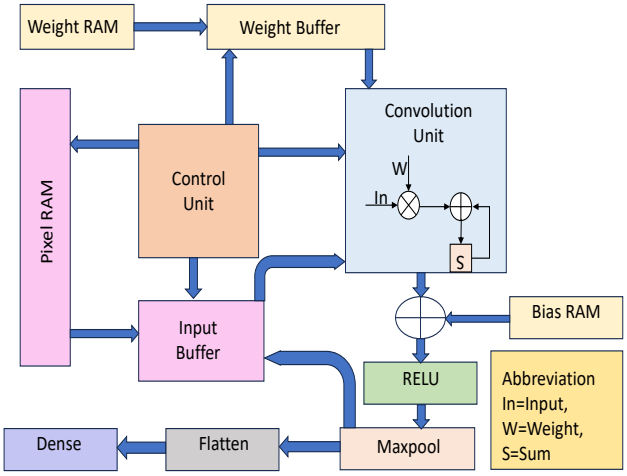

Figure 3: Dataflow Of Proposed Quantized CNN Model

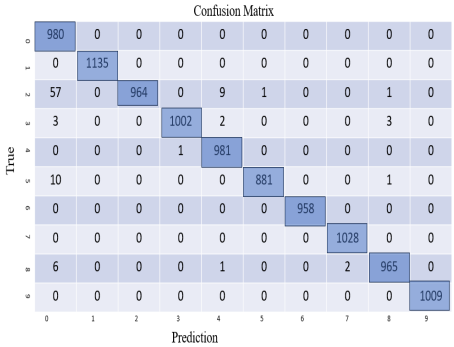

Figure 4: Confusion Matrix of Proposed Quantized CNN Model

Table 2: Resource Utilization Guideline in Vitis HLS

Criteria	Guideline	Actual	Status
LUT	70%	7.18%	OK
FD	50%	3.05%	OK
LUTRAM+SRL	25%	2.12%	OK
MUXF7	15%	0.79%	OK
DSP	80%	9.09%	OK
RAMB/FIFO	80%	8.57%	OK
DSP+RAMB+URAM (Avg)	70%	8.83%	OK
BUFGCE* + BUFGCTRL	24	0	OK
DONT_TOUCH (cells/nets)	0	0	OK
MARK_DEBUG (nets)	0	0	OK
Control Sets	998	157	OK
Average Fanout for modules 100k cells	4	0	OK
Non-FD high fanout nets 10k loads	0	0	OK

However, the various HLS optimizations used has reduced the usage of LUTs, DSP, BRAM, and slices.

Table 4 presents the on-chip power consumption analysis, revealing that the board consumes a total power of 1.72W. The primary

Table 3: Resource Utilization

Name	Used	Available
Slice Luts	6752	53200
Slice Registers	7165	106400
Slice	2634	13300
LUT	6150	53200
BRAM	17	140
DSP	20	220

contributor to this power consumption is the ZYNQ 7000 SoC processing system, which accounts for 1.527W. Additionally, BRAM consumes 0.005W of power, attributed to the storage of variables, weights, and biases. The remaining 0.139W is consumed by static power, which is mainly due to factors such as leakage currents, on-chip memory, and clock networks.

Table 4: Total On-Chip Power Consumption

Name	Power(W)
Clocks	0.013
Signals	0.014
Logic	0.014
BRAM	0.005
DSP	0.008
PS7	1.527
Static	0.139
Total Power	1.720

The PS-PL interfacing setup is depicted in Fig. 5, showcasing the connectivity between components. In this configuration, our ARM processor establishes connections with both our CNN IP and AXI DMA modules via the AXI LITE interface [16]. This arrangement empowers the ARM processor to efficiently manage the operations of the CNN IP and AXI DMA by issuing commands as required. Furthermore, the ARM processor interfaces with the DDR memory through dedicated data lines, facilitating seamless data exchange.

The AXI DMA [7] is a crucial bridge between our CNN IP and the DDR or BRAM memory. It facilitates stream data conversion into memory-mapped format, allowing storage in DDR or BRAM memory. Conversely, it converts memory-mapped data into stream data, enabling seamless interaction with our CNN IP. Notably, our CNN IP relies on streamed data for its operations, leveraging FIFO streams internally for efficient data flow across various functions. The communication between PS-PL sections is illustrated in 5.

The Block Design of proposed CNN model is presented in Fig. 6 illustrating the integration between the ZYNQ processing system and various programmable logic IPs. In this configuration, the Convolutional Neural Network (CNN) IP receives input data from memory via the M AXIS MM2S port of the AXI DMA IP [13]. Once processed, the CNN IP transfers its output to the Middle data IP, which incorporates the TLAST and TKEEP signals. Subsequently, the Middle data IP [14] forwards its output to the S AXIS S2MM

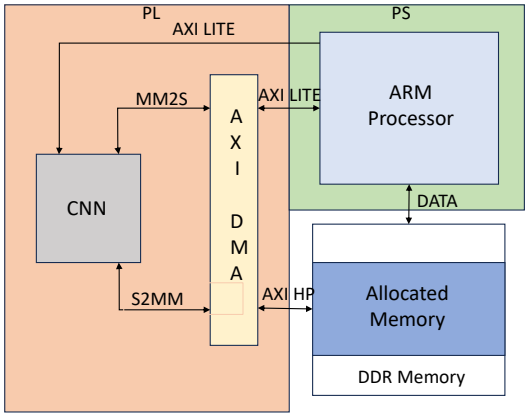

Figure 5: PS-PL Interfacing[3]

port of the AXI DMA IP [7], ultimately writing the processed output back into memory.

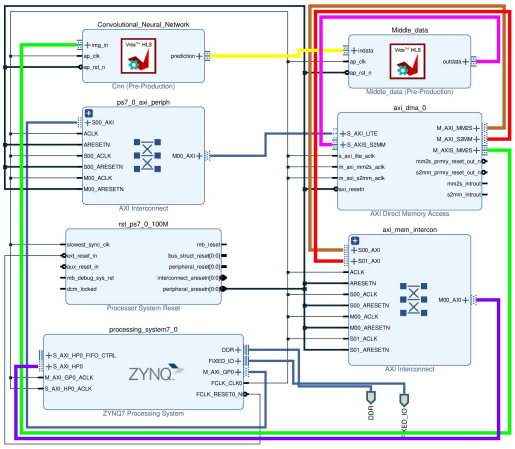

Figure 6: Block Design of Quantized CNN Model

Additionally, Fig. 6 provides the implemented design of the CNN model, while Fig. 8 offers insight into the routing aspect of the system. Fig. 7 illustrates the experimental setup, demonstrating the connection of the PYNQ Z2 Board to a laptop via a router. The setup involves matching the laptop's IP address with that of the PYNQ Z2 Board and establishing a connection with the PYNQ's processing system through Jupyter Notebook.

Within the Jupyter Notebook, we utilize Overlay to call the ".bit" file and transfer and receive data using the AXI DMA channel.We give input through the AXI_DMA_snd channel whereas the predictions are obtained through the AXI_DMA_rcv channel.

5 DISCUSSION

In this section, we compare our results with the [15] findings. Table 4 illustrates the total on-chip power consumption of the reference model, which amounts to 2.077W. In contrast, our model exhibits a lower power consumption of 1.72W. Notably, our model utilizes

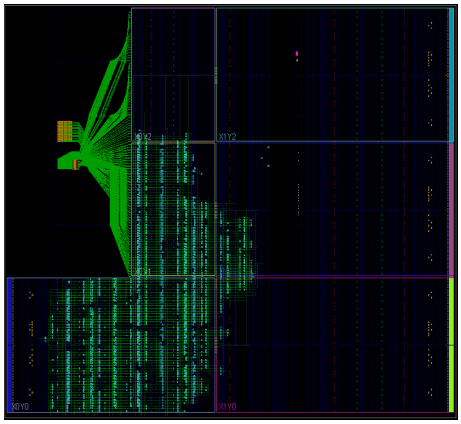

Figure 7: Routing Of Implemented Quantized CNN Model

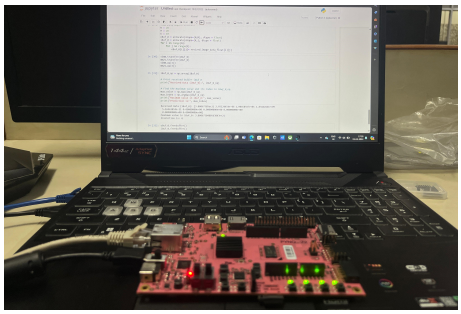

Figure 8: Experimental Setup

Table 5: Comparison table of Power consumption and resource utilization

On-Chip Resources	Summary of Power consumption(W)		Used		Resource Utilization(%)	
	[15]	Our	[15]	Our	[15]	Our
Clocks	0.092	0.013	3	3	-	-
Slice Logic	0.138	-	9446	2634	71.02	19.8
LUT as logic	0.128	0.014	30585	6752	57.49	12.69
Register	0.006	-	51419	7165	48.33	6.73
BRAM	0.14	0.005	98.5	17	70.36	12.14
DSPs	0.045	0.008	65	20	29.55	9.09
Static Power (W)	0.164	0.139				
Total	2.077	1.72				

significantly fewer BRAM resources, with only 17 units compared to the reference model's 98.5 BRAM units.

Furthermore, our model demonstrates reduced resource utilization in terms of LUTs and Registers. Specifically, our model utilizes 6752 LUTs and 7165 Registers, which are notably lower than the reference model's utilization of 30585 LUTs and 51419 Registers. The proposed optimized model computes 100 images in 0.2404 seconds, significantly faster than the [15] 0.87 seconds.

6 CONCLUSION AND FUTURE SCOPE

In this paper, an approach for image recognition is proposed, leveraging Convolutional Neural Networks (CNNs) accelerated by Field-Programmable Gate Arrays (FPGAs). An optimized model with fewer filters, resulting in reduced memory usage while achieving higher accuracy is trained, and the optimized architecture simulated and sythesized in HLS C++ code, incorporating techniques such as pipelining as well as inlining and array partioning. Moreover, our resource utilization is substantially lower due to memory and directive optimizations, incorporating techniques such as quantization. The FPGA's fast and parallel computing capabilities facilitate efficient algorithm execution and parallel computations, yielding desired results. However, there is still room for improvement in both algorithm design and memory optimization.

REFERENCES

[1] Sándor Tihamér Brassai, Attila Hammas, and Balázs Bustya. 2022. Framework for neural network hardware implementation. In *2022 23rd International Carpathian Control Conference (ICCC)*. 387–391. https://doi.org/10.1109/ICCC54292.2022.9805981

[2] Natalia Cherezova, Dmitri Mihhailov, Sergei Devadze, and Artur Jutman. 2022. HLS-based Optimization of Tau Triggering Algorithm for LHC: a case study. In *2022 18th Biennial Baltic Electronics Conference (BEC)*. 1–6. https://doi.org/10.1109/BEC56180.2022.9935599

[3] Advanced Micro Devices. 2002. PYNQ. Retrieved March 5, 2023 from https://pynq.readthedocs.io/en/latest/

[4] Advanced Micro Devices. 2024. AMD-Xilinx tools and software accessed from:. Retrieved April 10, 2024 from https://www.amd.com/en/corporate/university-program.html

[5] Advanced Micro Devices. 2024. Usage of PYNQ framework is available at:. Retrieved April 10, 2024 from https://www.pynq.io/

[6] Jiaming Huang, Junyan Yang, Saisai Nui, Hang Yi, Wei Wang, and Hai-Bao Chen. 2021. A Low-Bit Quantized and HLS-Based Neural Network FPGA Accelerator for Object Detection. In *2021 China Semiconductor Technology International Conference (CSTIC)*. 1–3. https://doi.org/10.1109/CSTIC52283.2021.9461256

[7] Adam Kidwell, Matthew P. Horvath, Darrin Hanna, and Bryant Jones. 2023. Lightweight Direct Memory Access on FPGA Using AXI Protocol. In *2023 Congress in Computer Science, Computer Engineering, & Applied Computing (CSCE)*. 881–886. https://doi.org/10.1109/CSCE60160.2023.00149

[8] Yaan LeCun. 1998. THE MNIST DATABASE of handwritten digits. Retrieved April 10, 2024 from http://yann.lecun.org/exdb/mnist/

[9] Xiao-Yang Liu, Zeliang Zhang, Zhiyuan Wang, Han Lu, Xiaodong Wang, and Anwar Walid. 2023. High-Performance Tensor Learning Primitives Using GPU Tensor Cores. *IEEE Trans. Comput.* 72, 6 (2023), 1733–1746. https://doi.org/10.1109/TC.2022.3222955

[10] Pranaav Jothi M, Manoj Kumar B, Subalakshmi Sarvani S, and S. P. Joy Vasantha Rani. 2023. Zynq SoC-based IPs for Deep Learning Neural Networks. In *2023 Winter Summit on Smart Computing and Networks (WiSSCoN)*. 1–5. https://doi.org/10.1109/WiSSCoN56857.2023.10133848

[11] Abdul Rahoof, Vivek Chaturvedi, and Muhammad Shafique. 2023. FastCaps: A Design Methodology for Accelerating Capsule Network on Field Programmable Gate Arrays. In *2023 International Joint Conference on Neural Networks (IJCNN)*. 1–8. https://doi.org/10.1109/IJCNN54540.2023.10191653

[12] Muhammad Sarg, Ahmed H. Khalil, and Hassan Mostafa. 2021. Efficient HLS Implementation for Convolutional Neural Networks Accelerator on an SoC. In *2021 International Conference on Microelectronics (ICM)*. 1–4. https://doi.org/10.1109/ICM52667.2021.9664920

[13] Tomyslav Sledevič. 2019. Synchronization of AXI Streaming Interfaces for Convolution Core Implementation on FPGA. In *2019 IEEE 7th IEEE Workshop on Advances in Information, Electronic and Electrical Engineering (AIEEE)*. 1–4. https://doi.org/10.1109/AIEEE48629.2019.8977065

[14] URI-nextlab. 2022. Parallel Programming. Retrieved April 10, 2024 from https://uri-nextlab.github.io/ParallelProgammingLabs/

[15] Rama Muni Reddy Yanamala and Muralidhar Pullakandam. 2022. An Efficient Configurable Hardware Accelerator Design for CNN on Low Memory 32-Bit Edge Device. In *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*. 112–117. https://doi.org/10.1109/iSES54909.2022.00033

[16] Haonan Zhang, Jinbo Wang, Lu Kong, Panpan Xue, and Zhaohui Yao. 2022. Design of a Convolutional Neural Network Accelerator based on PYNQ. In *2022 9th International Conference on Dependable Systems and Their Applications (DSA)*. 133–138. https://doi.org/10.1109/DSA56465.2022.00025